```
      if (!perms.contains(permission)) {
        halt(403);    #E
      }
    };
  }
```

#A Return a new Spark filter as a lambda expression. You can access the arguments to the outer method call inside the lambda expression.
#B Ignore requests that don’t match the request method.
#C First call the existing filter to ensure the user is authenticated.
#D You look up permissions for the current user in the given space, defaulting to no permissions.
#E If the user doesn’t have permission, then abort with a 403 Forbidden status.

3.6.3 Enforcing access control in Natter

You can now add filters to each operation in your main method, as shown in listing 3.7. Before each Spark route you add a new `before()` filter that enforces correct permissions. Each filter path has to have a `:spaceId` path parameter so that the filter can determine which space is being operated on. Open the Main.java class in your editor and ensure that your `main()` method matches the contents of listing 3.7. New filters enforcing permission checks are highlighted in bold.

Listing 3.7 Adding authorization filters